Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_FRAMES 3

void printFrames(int frames[], int n)
{
    for (int i = 0; i < n; i++)
    {
        if (frames[i] == -1)
        {
            printf(" - ");
        }
        else
        {
            printf(" %d ", frames[i]);
        }
    }
    printf("\n");
}

int main()
{
    int frames[MAX_FRAMES];

    // Initialize frames to -1 (empty)
    for (int i = 0; i < MAX_FRAMES; i++)
    {
        frames[i] = -1;
    }

    int pageFaults = 0;

    int referenceString[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int n = sizeof(referenceString) / sizeof(referenceString[0]);

    printf("Reference String: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", referenceString[i]);
    }
    printf("\n\n");

    printf("Page Replacement Order:\n");

    for (int i = 0; i < n; i++)
    {
        int page = referenceString[i];
        int pageFound = 0;

        // Check if page already exists
        for (int j = 0; j < MAX_FRAMES; j++)
        {
            if (frames[j] == page)
            {
                pageFound = 1;
                break;
            }
        }

        if (!pageFound)
        {
            printf("Page %d -> ", page);

            int placed = 0;

            // FIRST: check for empty frame
            for (int j = 0; j < MAX_FRAMES; j++)
            {
                if (frames[j] == -1)
                {
                    frames[j] = page;
                    placed = 1;
                    break;
                }
            }

            // If no empty frame → apply optimal
            if (!placed)
            {
                int optimalPage = 0;
                int farthestDistance = -1;

                for (int j = 0; j < MAX_FRAMES; j++)
                {
                    int futureDistance = 0;
                    int found = 0;

                    for (int k = i + 1; k < n; k++)
                    {
                        if (referenceString[k] == frames[j])
                        {
                            found = 1;
                            break;
                        }
                        futureDistance++;
                    }

                    if (!found)
                    {
                        optimalPage = j;
                        break;
                    }

                    if (futureDistance > farthestDistance)
                    {
                        farthestDistance = futureDistance;
                        optimalPage = j;
                    }
                }

                frames[optimalPage] = page;
            }

            printFrames(frames, MAX_FRAMES);
            pageFaults++;
        }
    }

    printf("\nTotal Page Faults: %d\n", pageFaults);

    return 0;
}
```

Output:

```
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:
Page 7 ->  7  -  -
Page 0 ->  7  0  -
Page 1 ->  7  0  1
Page 2 ->  2  0  1
Page 3 ->  2  0  3
Page 4 ->  2  4  3
Page 0 ->  2  0  3

Total Page Faults: 7

Process returned 0 (0x0)   execution time : 0.959 s
Press any key to continue.
```